

中山大學

SUN YAT-SEN UNIVERSITY

数据库原理实验报告

二次元周边商品店数据库重构与应用实现

姓 名　郑立峰　李维政

学 号　24325341　24325143

学 院　计算机学院

专 业　计算机科学与技术

2026 年 10 月 24 日

目录

1 实验目标与阶段安排 1
1.1 从关系数据库到可操作应用 . . . 1
1.2 总体思路与实现范围 . . . 1
1.3 实验环境 . . . 1

2 第 5 周 ER 模型与业务规则检查 1
2.1 实体划分与联系表达 . . . 1
2.2 商品与库存的基数和参与约束 . . . 2
2.3 销售订单与人员联系 . . . 3
2.4 预售与采购规则及原结构问题 . . . 4

3 第 6 周函数依赖与规范化取舍 5
3.1 候选码与主要函数依赖 . . . 5
3.2 规范化示例与分解性质 . . . 6
3.3 保留字段的设计理由 . . . 6
3.4 需求快照与到货记录 . . . 7

4 迁移方案与回归结果 8
4.1 三项结构调整及执行顺序 . . . 8
4.2 冲突处理与实际调试 . . . 8
4.3 迁移前后语义保持 . . . 9

5 第 7–9 周应用设计与实现 10
5.1 应用组织与分层 . . . 10
5.2 查询与写入的执行过程 . . . 11
5.3 身份与三层权限控制 . . . 12
5.4 草稿与结账的事务边界 . . . 13

6 本机业务流程与结果分析 14
6.1 商品维护与库存入库 . . . 14
6.2 草稿编辑，结账与数据库核对 . . . 15
6.3 两位顾客的本人订单查询 . . . 16
6.4 库存不足与整单回滚 . . . 17

7 集成测试与结果分析 18

8 分工，AI 辅助与实验总结 19
8.1 小组分工与辅助过程 . . . 19
8.2 实验认识与当前不足 . . . 19

1 实验目标与阶段安排

1.1 从关系数据库到可操作应用

第一阶段已经在 SQL Server 中建立了商品，库存和订单等数据。第 5–9 周继续把这些记录连成实际业务，先检查表结构怎样表达店铺规则，再补齐约束和操作流程，最后通过网页完成销售，并到数据库核对结果。

1.2 总体思路与实现范围

这次设计从已有业务结构出发。ER 检查先把商品，订单和人员之间的关系整理清楚，函数依赖分析再判断字段为什么这样保存。第六周据此保留 15 张业务表和 96 个字段，将迁移集中在三项采购约束上，应用开发便可以沿用这些对象继续组织操作。

应用选择现货销售作为完整流程，商品建档后入库，店员登记商品与数量，确认结账，顾客再查询本人订单。围绕这笔销售，商品资料和库存数量分开维护，登记单价保留顾客确认的价格，库存扣减与订单完成放在同一事务中。页面再按店长，店员，库存员和顾客的职责安排入口，每个角色看到的内容对应自己的工作。

网页登录通过新增的 `WebUser` 表绑定员工或会员，另加两个编号序列，五类受限数据库身份和顾客订单行级过滤。这样既能承接原有业务资料，也能确定操作由谁发起。原有预售和采购数据继续保留，履约与验收界面留待后续扩展。

1.3 实验环境

应用使用 Python 3.12 和 Flask，通过 pymssql 连接 SQL Server，运行环境由 Conda 管理。实验另建 `AnimeGoodsStoreDB_Week6App`，依次装入基础数据，迁移和应用安全结构。后面的操作主要在网页完成，SSMS 用来核对同一笔业务实际保存了什么。

2 第 5 周 ER 模型与业务规则检查

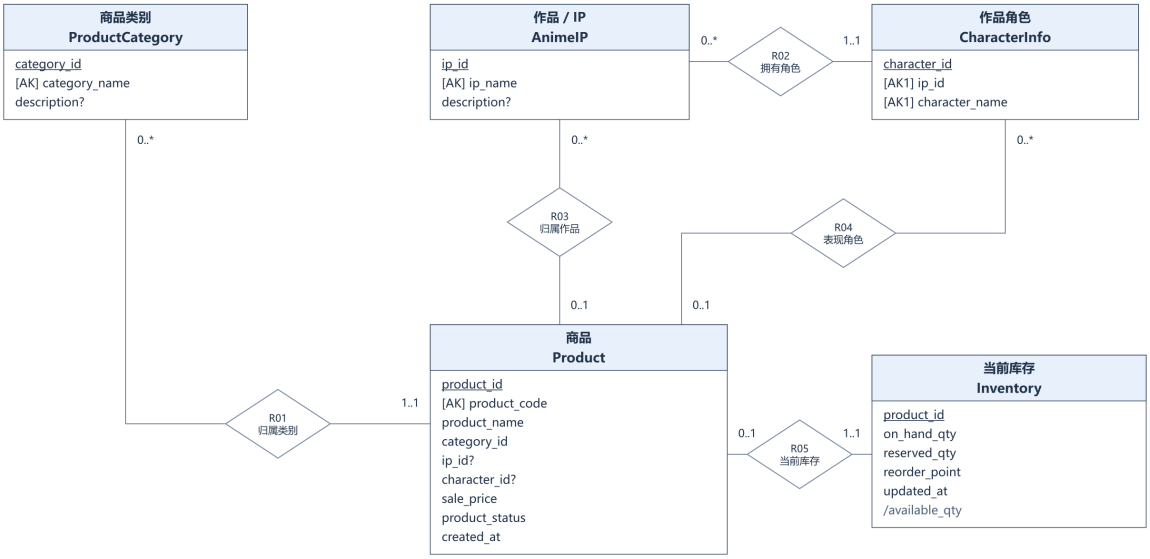
2.1 实体划分与联系表达

这次 ER 建模从已经存在的 15 张表往回检查，先确认每张表表示的业务对象，再看对象之间怎样联系。商品类别，IP 和人员资料可以独立存在，一笔订单则通过明细记录所购商品。这些联系既给出了连线的依据，也为后面检查唯一性，参与约束和业务状态提供了线索。

2.2 商品与库存的基数和参与约束

群像商品主要对应一部作品，具体角色商品则可以同时登记作品和角色，所以 IP 与角色按商品情况填写。填入角色时，组合外码会核对它所属的 IP，两项资料因此保持一致。每件商品还须归属一个类别，供目录分类和查询使用。

库存以 `product_id` 同时作为主码和外码，所以一件商品至多对应一条当前库存。原结构允许商品已经建档而库存行尚未建立，网页中再遇到这种情况，就要区分没有库存记录与库存为零。应用因此在新增商品的同一事务里补建零库存行，后续入库便能从明确的零开始。



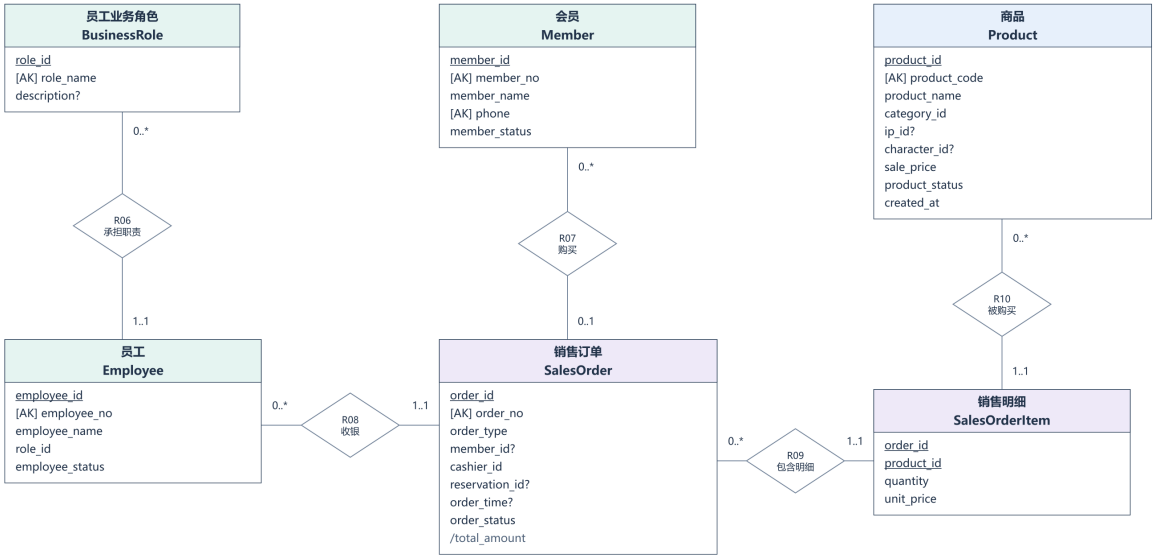
商品与库存模块 ER 图

图中用最小和最大次数标记参与联系的范围，下划线标记主码，AK 辅助标记候选码，问号和斜线分别表示可空与派生属性。可售数量由实际库存减去预留量得到，查询时计算，库存变化后就能按同一口径得到当前可售量。

2.3 销售订单与人员联系

销售订单先记录收银员工，购买者可以是会员，也可以是散客。具体商品放在明细中，以 `order_id,product_id` 共同定位一行。同一商品再次加入同一订单时，修改这一行的数量即可。数量和登记单价共同描述这笔订单中的购买情况，因此由订单与商品一起确定。

订单与明细的关系还要结合状态理解。创建草稿时允许暂时没有商品，结账则要求至少一条明细。外码负责维持明细与订单的关联，结账检查接着核对订单状态和内容，成功后再将明细转为只读。这几项检查分别对应建单，编辑和完成的不同阶段。



销售，明细与人员模块 ER 图

订单中的收银员工还要与登录身份对应。业务角色表保存员工职责，网页登录后核对员工的启用状态和账户绑定职责，收银操作再沿用这个身份，订单中便能留下实际经办者。

2.4 预售与采购规则及原结构问题

会员取消预订后还可能重新预订，这两次记录需要分别保留。因此预订使用各自的编号标识，同一活动与会员的组合可以重复，不能直接作为候选码。提货再核对原会员和商品，一条预订最多形成一笔提货订单，使后续交易能够追溯到原来的预订。

采购采用一场预售最多一张采购单的约定，普通补货单的活动编号为空。检查原结构时，发现同活动多单和补货关联活动等记录仍可能通过已有约束，因此将这些差异整理出来，分别确定后续处理的位置。

编号	原结构问题	后续处理与保留的问题
P01	同一预售可能关联多张采购单。	第六周为非空活动编号增加筛选唯一索引，限制同活动最多一张采购单。
P02	状态包含部分到货，与整单到货约定不一致。	删除 `PARTIAL_RECEIVED` 状态，保留明细到货数量，数量与整单状态的联动仍未强制。
P03	完成订单可能无明细或继续被修改。	应用允许草稿编辑，结账拒绝空单，完成后交易内容只读。
P04	商品建档后可能没有库存行。	应用新增商品时同事务创建零库存行，库存调整也处理缺失记录。
P05	普通补货可能错误关联预售活动。	第六周以双向 CHECK 同时约束采购类型和活动编号。
P06	预售会员，商品，预订状态存在跨表规则。	留待完整预售履约接口处理。
P07	销售，库存与操作身份可能脱节。	当前现货结账通过登录身份，角色检查及统一事务实现，采购和预售链尚待扩展。

表 1: 由 ER 反查得到的问题与落实位置

同一活动的采购单数量可以用唯一限制控制，采购类型与活动关联可以用 CHECK 检查。结账涉及订单状态，人员和库存，需要在一次操作中联合处理。ER 检查得到的问题由此接到后续实现，结构约束先落实清晰的记录规则，业务层再组织跨表流程。

3 第 6 周函数依赖与规范化取舍

3.1 候选码与主要函数依赖

候选码先根据业务上的唯一性约定和已有约束确定，再检查各属性依赖单个码，整组码，还是某个非主属性。样例数据用于核对实际记录，依赖关系则以这些规则为依据，后面的部分依赖和传递依赖分析也由此展开。

关系	主要候选码	依赖与分析结论
`ProductCategory`	类别 ID，类别名称	任一候选码确定描述等属性，类别名称单独存储，避免在商品行内反复更新。
`CharacterInfo`	角色 ID，IP 与角色名称组合	角色 ID 确定 IP 及角色名称，辅助唯一组合 `character_id,ip_id` 是超码，但不是最小候选码。
`Product`	商品 ID，商品编号	商品码确定商品资料，在指定角色的子集内还存在角色 ID 决定 IP 的依赖，保留受控冗余。
`Inventory`	商品 ID	商品 ID 确定当前数量，预留数量，警戒值和更新时间，可售数量由查询计算。
`SalesOrder`	订单 ID，订单编号	订单码确定会员，收银员及状态，非空预订编号的条件唯一性不作为全表普通候选码。
`SalesOrderItem`	订单 ID 与商品 ID 组合	整组码确定数量和成交单价，不将商品当前售价误作历史成交价的决定因素。
`PresaleReservation`	预订 ID，预订编号	保留取消与重订历史，活动与会员组合不保证全表唯一。
`PurchaseOrderItem`	采购 ID 与商品 ID 组合	整组码确定采购量，成本和到货量，数量约束与状态联动需要分别判断。

表 2: 代表性关系的候选码与依赖分析

会员手机号具有唯一约束，可以据此分析候选码，员工手机号和供应商名称则按各自已有的标识规则判断。订单明细中，商品编号确定商品资料，数量和登记单价还与具体订单有关。因此商品资料独立保存，购买数量和价格留在明细中，目录调整后仍可追溯原交易。

3.2 规范化示例与分解性质

为了说明现有商品表与类别表为什么分开，先在理论上把它们合成一个关系。

$$R(\mathit{product_id}, \mathit{product_name}, \mathit{category_id}, \mathit{category_name})$$

商品 ID 决定商品名与类别 ID，类别 ID 又决定类别名。如果每件商品都带着一份类别名称，改一次类别名就要改多行。还没有商品的类别难以单独插入，最后一件商品被删除时，类别本身也可能跟着丢失，更新异常便由这种重复保存产生。

把商品资料与类别资料分开以后，得到下面两个关系。

$$R_1 = (\mathit{product_id}, \mathit{product_name}, \mathit{category_id})$$
$$R_2 = (\mathit{category_id}, \mathit{category_name})$$

两个关系共有类别 ID，类别 ID 能够决定 R_2 的全部属性，所以按类别 ID 连接时，可以把商品与所属类别重新对应起来，这个分解是无损的。原来的两组依赖分别留在 R_1 和 R_2 内部，检查可以在各自关系中完成，因此依赖也得到保持。

这个合并关系用于解释现有结构的分解性质。原库中的商品和类别已经分开，规范化检查支持继续保留这部分设计，第六周的实际迁移集中处理采购约束中发现的问题。

3.3 保留字段的设计理由

商品同时保留 IP 和具体角色。群像商品通过 IP 表示所属作品，具体角色商品再填写角色，两个字段因此有各自的用途。在角色非空的记录中，角色 ID 也能决定 IP，这就带来了一份重复信息。角色 ID 不是商品表超码，IP 也不是主属性，这部分条件依赖不满足严格第三范式。保留这两个字段能够覆盖上述商品情况，组合外码负责核对角色归属，维护这份冗余是相应的代价。

保留历史值，派生值尽量查询计算。目录价描述商品当前的售价，明细单价记录这笔购买确认的价格。把登记单价保存在订单中，目录以后调价时，历史交易仍有原来的依据。订单总额由明细数量乘单价求和，可售量由实际库存减预留量得到，这两项随查询计算，维护时只需保证来源记录和计算口径一致。

3.4 需求快照与到货记录

预售需求也有类似的时点区别。采购决策时保存一份需求快照，之后的取消或重订改变当前需求，当时的决策依据仍能留下。计划采购量还会结合已有库存等情况确定，因此作为经营决定单独保存，需求快照和额外备货量供判断时参考。

订金实收和退款额按实际收付保留。当前设计将它们作为已经发生的金额记录，保留原值能够追溯收付情况。若以后建立实收额始终等于数量乘固定订金单价的规则，再依据这项规则分析可计算冗余。

`received_qty` 保留实际记录的到货量，现有约束将它限定在零到采购量之间。整单状态与每行到货量的联动仍待完成，保留字段可以继续表达当前的收货情况。若后续实现严格的整单收货，再结合完整流程判断到货量是否能够由状态和采购量推导。

选择	采用理由	还需保证什么
保留商品 IP 与角色	支持群像商品及角色商品的不同表达。	校验角色归属 IP，避免不匹配。
保留明细成交价	保存交易历史，允许之后调整商品售价。	登记时保存目录标价，结账沿用，完成后禁止修改。
查询计算总额与可售量	数值由现有记录确定，避免重复维护。	查询与页面使用一致计算口径。
保留需求决策快照	保存采购决策时依据，便于追溯。	区分快照与当前汇总值。
暂保留采购到货量	保持原数据与操作含义。	后续实现数量与收货状态的事务联动。

表 3: 结构保留与受控冗余的取舍

这些取舍使 15 张业务表继续保留，实际调整集中在采购规则上。规范化分析在这里帮助区分了独立资料，历史事实和派生值，也说明了每份冗余的用途及维护责任。

4 迁移方案与回归结果

4.1 三项结构调整及执行顺序

先留一份未迁移的基线，再在相同数据的副本上执行调整，后面才能对照变化。脚本检查旧结构与冲突记录，满足前提后才在一个事务中建立索引和替换 CHECK，冲突时列出记录并停止，保留原数据供核对。

```
CREATE UNIQUE INDEX UX_PurchaseOrder_PresaleID
ON dbo.PurchaseOrder(presale_id)
WHERE presale_id IS NOT NULL;

ALTER TABLE dbo.PurchaseOrder WITH CHECK
ADD CONSTRAINT CK_PurchaseOrder_TypePresale CHECK
((purchase_type='PRESALE' AND presale_id IS NOT NULL)
 OR (purchase_type='RESTOCK' AND presale_id IS NULL));

ALTER TABLE dbo.PurchaseOrder WITH CHECK
ADD CONSTRAINT CK_PurchaseOrder_Status CHECK
(purchase_status IN ('CREATED','ORDERED','RECEIVED','CANCELLED'));
```

普通补货没有活动编号，唯一索引因此只检查非空的 presale_id，多张补货单仍可保存。已取消的采购单也受它限制，所以同一活动最多一张采购记录，取消后另建采购单仍会被拒绝。旧约束移除与回滚由完整迁移脚本处理。

采购类型与活动编号要成对检查，预售采购关联活动，普通补货的活动编号为空，两种记录各有明确条件。状态域同时移除部分到货，落实整单到货的状态约定。明细到货量与整单状态的同步则留待验收流程继续处理。

建立约束时使用 WITHCHECK 检查既有行，脚本同时启用 XACT_ABORT 和筛选索引所需选项。旧结构不符或已迁移过时就停止，避免在错误前提下改表。

4.2 冲突处理与实际调试

同活动多单，类型关联不符和非法状态都会使迁移停止，原记录仍保留，已迁移的副本再次执行则返回 51005。调试中还实际遇到过 1934 错误，原因是筛选索引要求的会话选项不一致。补齐 SET 并关闭 NUMERIC_ROUNDABORT 后，45 项回归通过。索引建立以后，后续写入仍受这些选项影响，所以应用连接也沿用相同设置。

4.3 迁移前后语义保持

回归分类	通过数	检查内容
DATA	15	全部业务表的行数及全列双向 EXCEPT，检查数据是否增减或改变。
QUERY	5	订单明细，金额，销量，库存等关键查询，检查业务结果是否保持。
META	7	旧结构与目标结构的键，索引，约束等元数据。
TARGET	1	新增筛选唯一索引和两项 CHECK 均启用且受信任。
WRITE	4	合法写入仍然可以执行。
REJECT	7	重复活动采购，类型不符，非法状态等反例被拒绝。
ROLE	6	既有员工角色的允许与拒绝行为。
合计	45	45 项通过，0 项失败。

表 4: 第六周独立副本回归结果

核对项目	迁移前	迁移后	解释
SO004 金额	124.00	124.00	两条明细汇总保持。
6 笔订单金额合计	530.00	530.00	关键查询语义保持。
6 件商品可售库存合计	42	42	数量为 8，5，2，12，6，9。

表 5: 固定样例的迁移前后对照

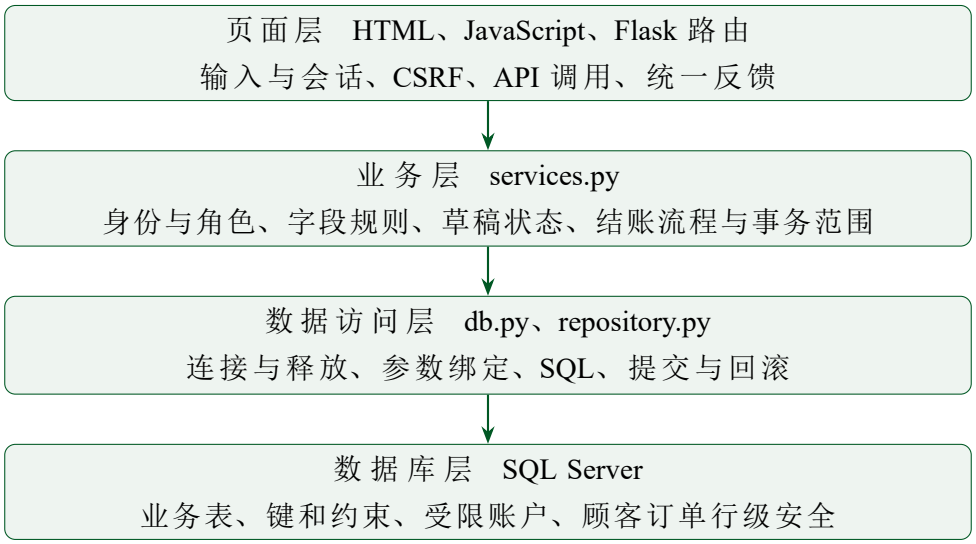
迁移后 SO004 为 124 元，6 笔订单合计 530 元，可售库存合计 42 件，与基线逐项一致。全部表的全列双向 EXCEPT 继续核对原数据，合法写入和非法输入测试再分别检查保留的能力与新增限制。查询结果，数据内容和约束行为合在一起，构成这次迁移的回归依据。

5 第 7–9 周应用设计与实现

5.1 应用组织与分层

确认结账需要同时知道登录者的职责，订单状态和当前库存，因此由后端集中判断。页面负责收集输入和显示结果，业务层把检查和修改组织成一次操作，数据访问层执行 SQL 并管理连接，各个入口由此共用相同的结账规则。

页面采用 Flask 模板配合少量 JavaScript，通过 JSON API 完成查询和表单提交。这些交互已经能够支撑商品维护，库存调整和收银，前端构建配置也较少，开发工作便可集中在角色，价格和事务等业务处理上。



应用分层及请求流向

模块	负责内容
`app/templates/index.html`	页面结构，表格与对话框。
`app/static/app.js`	查询，表单提交，角色入口和错误展示。
`app/__init__.py`	路由，会话，CSRF，统一响应与异常映射。
`app/services.py`	权限，输入与业务状态校验，组织完整业务操作。
`app/db.py`	建立同事务连接，提交，回滚和关闭。
`app/repository.py`	参数化 SQL 与结果读取。
`sql/01_web_security.sql`	登录技术表，受限角色与顾客行级过滤。

表 6: 各层在源代码中的位置

5.2 查询与写入的执行过程

以搜索商品为例，页面将输入的名称或编号交给 Flask 路由，后端确认登录身份和搜索词长度，再让 Repository 查询商品与库存。查询结果返回后刷新表格，SQL 和参数绑定统一放在数据访问层，后续写入也沿用这条调用路径。

```
# services.py
with self.connection(user) as repo:
    return repo.products("%" + term + "%", user["role"] in STAFF)
```

```
-- repository.py 中查询的参数绑定形式
WHERE (p.product_name LIKE %s OR p.product_code LIKE %s)
```

百分号和搜索词拼成 LIKE 的参数值，SQL 语句本身保持固定。输入中的引号也按参数内容处理。商品名，数量和价格等写入值采用同样的参数绑定方式，数据访问层统一管理语句与输入。

新增商品比查询多了一层需要一起完成的修改。资料通过编号，类别，角色归属和价格检查后，商品与零库存行要在同一事务里保存，失败时一起撤销。删除则要再看库存和历史引用，已经参与交易的商品用停售保留记录，避免历史订单失去原来的商品关联。

库存调整先根据增减量算出实际量，再检查它至少能够覆盖预留量，警戒值也在这一步保存。草稿登记时核对会员，商品，数量和页面标价，结账继续检查当前身份与订单状态。写入成功后重新查询结果，页面显示的便是数据库实际保存的状态。

这些操作的失败也由后端统一接收并转为页面提示。数量不足时保留草稿，店员可以调整后重试，连接失败则提示暂时不可用。响应与日志共用请求编号，后端另外保存异常细节，页面只显示当前需要采取的处理。

5.3 身份与三层权限控制

网页登录后先确定账户对应的员工或会员，再核对启用状态和职责。密码以哈希形式保存，后续业务从会话和数据库绑定中取得身份，订单经办人和权限检查便有了统一来源。

角色	商品	库存	订单
店长 `manager`	查询，新增，修改，条件删除。	查询与调整。	创建草稿，编辑明细，结账，取消及草稿删除。
店员 `clerk`	查询。	在目录查看可售量。	创建草稿，编辑明细，结账，取消及草稿删除。
库存员 `inventory`	查询。	查询与调整。	不开放订单和会员查询。
顾客 `customer`	查询在售商品。	仅随目录查看可售量。	查询本人订单，不开放经营写入。

表 7: 四类业务角色的主要操作范围

界面初版给多个角色同时提供了商品目录和库存管理，两处内容相近，检查时发现入口与职责需要进一步对应。修改后，店长在商品页维护资料与售价，在库存页处理数量，店员从目录查看可售量并进入收银，库存员集中使用库存管理，顾客浏览在售商品和本人订单。按实际工作安排页面以后，每个入口的用途更清楚。

权限检查沿着同一职责划分继续往下落实。后端逐项核对操作，再选用 `auth`，`manager`，`clerk`，`inventory` 或 `customer` 受限身份连接数据库，页面入口，业务检查和数据库授权各自承担一层控制。

顾客使用同一类角色，订单范围还要由绑定的会员决定。因此连接中设置只读 `member_id` 上下文，由 SQL Server 行级安全策略过滤订单和明细，API 也按会员筛选。角色确定可以进行什么操作，会员绑定进一步确定可以读取哪些记录。

页面校验能及时提示输入问题，后端仍要再检查一次。价格按十进制处理并限制正值，两位小数和上限，数量则拒绝零，负数，布尔值和非整数，写请求同时校验 CSRF。这样绕过表单直接提交请求时，仍然经过同样的规则。

5.4 草稿与结账的事务边界

订单先存为 CREATED 草稿，供店员调整会员和数量，库存留到结账时扣减。原实现会在改数量和结账时取最新售价，按线下销售场景重新核对后，将价格确定的时点放在首次登记。顾客先看目录标价再决定购买，登记时核对页面价格并保存单价，后续刷新，改数量和结账都沿用它，确认金额便能对应到最终记录。若首次登记时页面标价已过期，就提示刷新核对并拒绝本次写入。结账成功后转为 COMPLETED，保存成交内容。

结账先锁定订单，确认它仍处于草稿状态，会员有效且已经有明细，再按商品 ID 顺序检查在售状态与可售库存。所有明细共用一个事务，库存扣减和订单完成一起提交。这样遇到后续商品不足时，前面已执行的修改仍在同一事务中，可以随整单撤销。

```
UPDATE dbo.Inventory
SET on_hand_qty = on_hand_qty - %s,
    updated_at = SYSDATETIME()
WHERE product_id = %s
  AND on_hand_qty - reserved_qty >= %s;
```

库存是否足够也放进 UPDATE 的条件里，只有可售量满足数量时才扣减，再根据影响行数判断结果。这样读取库存以后即使还有其他订单参与竞争，写入时仍会重新受这个条件限制。商品锁按固定顺序取得，配合事务处理并发争用。

```
# db.py 中事务管理的核心结构
try:
    # connection 已建立，cursor 已设置所需 SET 选项
    yield Repository(cursor)
    connection.commit()
except BaseException:
    connection.rollback()
    raise
finally:
    connection.close()
```

这段事务管理把整笔销售作为提交单位。业务拒绝，约束错误和执行异常都进入回滚路径，已执行的库存扣减一并撤销，订单继续保持草稿和登记价格。成功时统一提交，最后关闭连接，调用者只需围绕完整操作处理结果。

6 本机业务流程与结果分析

6.1 商品维护与库存入库

这次先由店长新增 DEMO001，再把名称改为初音未来徽章，售价改为 120 元，类别，IP 和角色分别设为徽章，初音未来和初音未来。另一件 DEMO002 的售价为 20 元，名称实际保存成文本 None。保存后再回到目录查询，核对页面确实读到了修改后的资料，新增商品的库存则从零开始。



商品新增与修改表单，局部截图

接着由库存管理员入库，两件商品分别增加 10 件和 5 件，警戒值设为 3 和 2，查询得到可售量合计 15。商品建档先确定资料和售价，入库再确定实际数量，这两步分别对应店长和库存员的工作。店员随后从目录读取售价与可售量，用于登记订单。



入库后两件商品库存为 10 和 5，局部截图

6.2 草稿编辑，结账与数据库核对

店员为会员 M001 建立 WEB0000010015 草稿，加入两件 DEMO001 和一件 DEMO002，金额为 260 元。先把第一行改为 1 件并保存，金额随之变成 140 元，再恢复为 2 件。这一步同时检查数量修改与重新汇总，草稿期间库存仍保持原数，最后才确认结账。

确认以后，订单变为已完成，明细转为只读。两行金额分别为 240 元和 20 元，总额 260 元，库存从 10 和 5 变成 8 和 4，正好分别减少 2 件和 1 件。订单金额和库存变化与购买数量逐项对应。



结账后保存成交价格与订单明细，局部截图

页面显示成功之后，再到 SSMS 查询同一笔订单。关联商品，库存和明细后，状态为 COMPLETED，会员 ID 为 1，数量与单价也都和网页一致，网页上的操作结果由数据库记录得到核实。

Results Messages

	product_code	product_name	on_hand_qty	reserved_qty	available_qty
1	DEMO001	初音未来徽章	8	0	8
2	DEMO002	None	4	0	4

	order_no	order_status	member_id	product_code	quantity	unit_price	line_amount
1	WEB0000010015	COMPLETED	1	DEMO001	2	120.00	240.00
2	WEB0000010015	COMPLETED	1	DEMO002	1	20.00	20.00

数据库中的库存与成交明细，局部截图

6.3 两位顾客的本人订单查询

登录 `customer1`，列表中能看到刚完成的 260 元订单，以及 SO004 和 SO003。打开明细可以核对购买内容，收银留下的记录以只读形式提供给顾客。



顾客 1 可以查看会员 1 的订单，局部截图

换成 `customer2` 后，列表只有会员 2 的 SO002，金额 90 元。两个账号共用顾客界面，各自读取绑定会员的订单，页面结果符合本人记录的范围。跨会员直接访问再由后面的集成测试检查。



顾客 2 只显示会员 2 的订单，局部截图

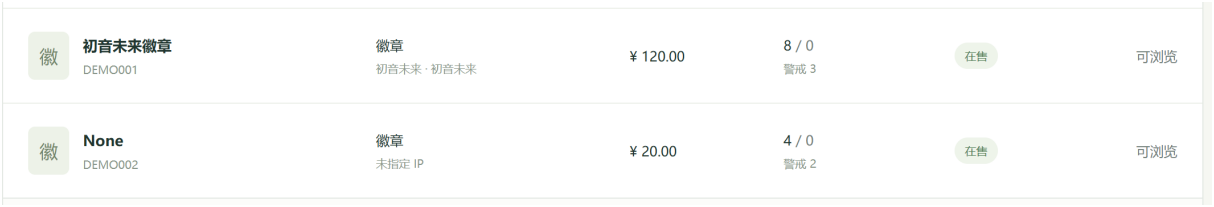
6.4 库存不足与整单回滚

接着另建 WEB0000010016，第一行买 1 件 DEMO001，第二行买 10 件 DEMO002，金额 320 元。库存分别为 8 和 4，所以第一行足够，第二行不足。特意让不足出现在后面，才能检查前面已经处理的商品是否也会撤销。确认后返回 INSUFFICIENT_STOCK 和 409，页面提示库存不足，订单仍是草稿。



库存不足时拒绝结账并保留草稿，局部截图

再查库存，两件商品仍为 8 和 4，与结账前逐项相同。第二件失败后，第一件的扣减也被撤销，说明前面执行的修改也包含在同一回滚范围中。日志可以按页面上的请求编号找到这次操作，便于继续核对失败原因。



失败后两件商品库存保持 8 和 4，局部截图

这笔失败草稿随后确认删除，再查询时该订单编号已不存在。草稿处在登记阶段，允许调整后重试或删除，完成订单则保留交易内容，便于顾客查询和后续核对。其他删除条件，引用冲突和越权情况由自动化测试继续检查。

7 集成测试与结果分析

最终执行 pytest 得到 48 项通过，0 失败，0 错误，0 跳过。其中 42 项连接真实 SQL Server 检查应用行为，另外 6 项检查密码重设辅助脚本。

测试范围	代表性检查	结果
登录与请求保护	登录退出，未登录访问，密码哈希与后端身份，CSRF 与非法 JSON。	通过
商品与库存	商品 CRUD，自动零库存，重复约束回滚，非法价格，预留保护，有库存和有引用商品删除拒绝。	通过
订单与事务	草稿及明细 CRUD，空单拒绝，目录标价核对，草稿锁价，多商品结账，历史价保持，库存不足整单回滚。	通过
角色与顾客隔离	API 越权拒绝，顾客跨会员访问与 SQL RLS，受限 SQL 账户直接执行拒绝。	通过
并发与故障	并发结账不超卖，数据库连接失败与日志，页面安全响应头。	通过
密码重设辅助	生成哈希匹配且不含明文，非法输入拒绝，已有输出文件保留。	通过

表 8: 本机 48 项测试的范围与结果

价格测试分别检查登记前后的处理。商品按 25 元登记以后把目录调为 30 元，刷新和改数量仍保留 25 元，两件结账 50 元，库存由 10 减到 8。首次登记若使用过期页面标价，集成测试得到拒绝写入和重新核对的提示。两种结果分别对应核对新登记的标价，以及保留已登记的约定价格。

参数化查询测试使用带引号和 SQL 片段的搜索词，结果返回空列表，随后商品仍可正常查询。结合 Repository 中的参数绑定方式，可以核对这些字符按搜索内容处理，查询语句结构保持固定。

权限测试直接请求顾客新增商品，店员调整库存和库存员查询订单，均得到拒绝。受限 SQL 账户执行越权语句时返回 229 错误。顾客跨会员访问同时检查 API 和数据库过滤，页面展示范围与直接请求的处理结果由此得到共同核对。

断连测试把连接指向不可用端口，页面得到 503，并能按请求编号找到日志。Windows 复跑时曾在读取这份日志处失败，原因是 UTF-8 日志被按默认编码读取。明确读取编码后全量复跑 48 项通过，断连响应和日志读取两处结果均完成核对。

并发用例设置实际库存 3 件，预留 1 件，让两个订单各买 2 件。结果一笔成功，另一笔返回库存不足，最终实际库存与预留量均为 1，恰好保留了预留的数量。该用例检查了两笔订单争用可售库存的情况，更大并发规模下的表现仍待压力测试。

8 分工，AI 辅助与实验总结

8.1 小组分工与辅助过程

郑立峰负责第 5–6 周的 ER 模型，规则检查，依赖与范式分析，以及采购约束迁移和回归材料。李维政在这个结构上完成第 7–9 周应用，连接数据库，区分角色操作，处理结账事务与异常，并进行集成测试和本机流程截图，再将各周结果整理进报告。最终提交前仍需共同核对图，代码和实际结果是否一致。

AI 参与了规则分析，代码和测试组织，连接排查，以及报告的整理。业务范围和字段取舍由小组确认，本机截图记录实际的角色切换，数量修改，结账和失败操作。自动测试的运行结果另行保留，报告里的价格，金额与库存都对应已有截图或查询结果，未用预设演示值替换实际观察，过程见 `ai_log.md`。

8.2 实验认识与当前不足

这一阶段的设计逐渐围绕完整业务展开。ER 图说明对象怎样关联，依赖分析说明字段承担什么含义，应用再把它们组织成能够执行的一次操作。唯一性和类型关联交给数据库约束，订单状态，人员职责和库存则在结账中联合检查，各部分顺着同一条业务流程接起来。

价格规则的修改使这种衔接更具体了。目录维护当前售价，明细保存登记时确认的价格，首次加入草稿时核对两者，之后按记录下来的价格结账。每份数据都有明确的用途和生效时点，保留历史单价的理由也从字段定义落到了实际收银过程。

库存失败案例进一步说明了验证应围绕整笔操作安排。第一件足够，第二件不足，回滚后两件库存都回到原值，测试因此观察到了前后修改怎样一起撤销。迁移前后对照，成功销售和失败回滚分别提供了不同侧面的依据，结论也随实际核对的范围确定。

现货流程目前已有页面，数据库记录和测试结果相互对应。后续扩展预售履约与采购验收时，还需把到货数量，整单状态和库存变化组织成完整操作，并补充相应的成功与失败验证。这部分继续作为当前实现的后续工作。